

8. String to Integer (atoi) Medium

Acceptance: 20.4% | [LeetCode](#)

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace** : Ignore any leading whitespace (" ").
2. **Signedness** : Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion** : Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding** : If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1:

Input: s = "42"

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

Step 2: "42" (no characters read because there is neither a '-' nor '+')

Step 3: "_42_" ("42" is read in)

Example 2:

Input: s = "-042"

Output: -42

Explanation:

Step 1: " -042" (leading whitespace is read and ignored)

Step 2: " _-042" ('-' is read, so the result should be negative)

Step 3: " _-042_" ("042" is read in, leading zeros ignored in the result)

Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)

Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')

Step 3: "_1337_ c0d3" ("1337" is read in; reading stops because the next character is a non-digit)

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

Step 1: "0-1" (no characters read because there is no leading whitespace)

Step 2: "0-1" (no characters read because there is neither a '-' nor '+')

Step 3: "_0_ -1" ("0" is read in; reading stops because the next character is a non-digit)

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

Constraints:

- $0 \leq s.length \leq 200$
- s consists of English letters (lower-case and upper-case), digits (0-9), ' ', '+', '-', and '.'.

Claude Opus Solution

String to Integer (atoi)

1. Intuition

The key insight is that this problem is really about **careful, sequential state processing**. We must process the string in a strict order of rules:

[whitespace] → [optional sign] → [digits] → [stop at non-digit]

The trickiest part isn't the parsing—it's the **overflow handling**. Since we're simulating a 32-bit signed integer, we must clamp the result to $[-2^{31}, 2^{31} - 1]$. In Python, integers are unbounded, so we don't get overflow naturally; we must check the bounds manually.

The pattern here is **finite-state machine (FSM) thinking**: at each character, we know exactly which "phase" we're in and what's allowed next.

2. Approach (Pseudocode)

1. Strip leading whitespace.
2. If string is empty after stripping → return 0.
3. Check first char:
 - if '-' → sign = -1, advance pointer
 - if '+' → sign = +1, advance pointer
 - else → sign = +1
4. While current char is a digit:
 - answer = answer * 10 + digit_value
 - advance pointer
5. Apply sign: answer *= sign
6. Clamp answer to [MIN_INT, MAX_INT].
7. Return answer.

3. Python Solution

```
class Solution:
    def myAtoi(self, s: str) -> int:
        # 32-bit signed integer bounds
        MAX_INT, MIN_INT = 2147483647, -2147483648 # 2^31 - 1 and -2^31

        answer = 0      # accumulated numeric value
        index = 0        # current reading position
        sign = 1         # default sign is positive

        # --- Step 1: Skip leading whitespace ---
        # strip() removes leading/trailing spaces. We only need leading,
        # but trailing spaces won't affect parsing since we stop at non-digits.
        s = s.strip()

        # --- Step 2: Empty after stripping means nothing to parse ---
        if len(s) == 0:
            return 0

        # --- Step 3: Determine the sign ---
        # Check the first character for an explicit sign.
        if s[0] == "-":
            sign = -1
            index += 1      # consume the sign character
        elif s[0] == "+":
            sign = 1
            index += 1      # consume the sign character
        # else: no sign present, sign stays +1, index stays 0

        # --- Step 4: Read consecutive digits ---
        # Build the number digit by digit.
        # Leading zeros are handled naturally: 0*10 + d = d.
        while index < len(s) and s[index].isdigit():
            digit = ord(s[index]) - ord("0") # convert char → int via ASCII
            answer = answer * 10 + digit      # shift left and add new digit
            index += 1
            # Optional early-exit optimization (see note below).

        # --- Step 5: Apply the sign ---
        answer *= sign

        # --- Step 6: Clamp to the 32-bit signed range ---
        # If answer > MAX_INT → return MAX_INT
        # If answer < MIN_INT → return MIN_INT
        answer = min(MAX_INT, max(answer, MIN_INT))
```

```
return answer
```

Note on overflow safety

In Python, `answer` can grow arbitrarily large, so we clamp **once** at the end. In languages like C++/Java, the integer would overflow *during* accumulation, so you'd need to check bounds **inside the loop**. Here's the language-agnostic safe version of the loop:

```
while index < len(s) and s[index].isdigit():
    digit = ord(s[index]) - ord("0")
    # Check overflow BEFORE multiplying to avoid wrap-around.
    if answer > (MAX_INT - digit) // 10:
        return MAX_INT if sign == 1 else MIN_INT
    answer = answer * 10 + digit
    index += 1
```

This is good to mention in an interview to show you understand real overflow mechanics.

4. Complexity Analysis

Metric	Value	Explanation
Time	$O(n)$	We scan each character at most once (strip + single pass through digits).
Space	$O(1)$	Only a constant number of variables. (Python's <code>strip()</code> creates a new string of $O(n)$, but conceptually the algorithm uses constant extra space.)

Strictly: `s.strip()` allocates a new string $\rightarrow O(n)$ space. To make it truly $O(1)$, use a manual whitespace-skip pointer instead of `strip()`.

5. Alternative Approaches

A. Manual pointer (no `strip()`) — true $O(1)$ space

Instead of `s.strip()`, walk an index past spaces:

```
n = len(s)
i = 0
while i < n and s[i] == ' ':
    i += 1
```

Avoids allocating a new string—cleaner for strict memory constraints.

B. Regular Expression

A one-liner using regex captures the valid prefix:

```
import re
def myAtoi(self, s: str) -> int:
    m = re.match(r'\s*([+-]?\d+)', s)
    if not m:
        return 0
    num = int(m.group(1))
    return max(-2**31, min(2**31 - 1, num))
```

- **Pros:** Concise.
- **Cons:** Hides the logic; interviewers usually want the manual FSM to test your character-handling skills.

C. Formal Finite-State Machine (FSM)

Define explicit states (`START`, `SIGNED`, `IN_NUMBER`, `END`) and a transition table mapping (state, char-type) $\rightarrow$ new state. Overkill for this problem but excellent for showing scalable parser design and handling complex grammar rules cleanly.

Key Takeaways

- **Order matters:** whitespace $\rightarrow$ sign $\rightarrow$ digits is a strict sequence.
- **Overflow is the real test:** clamp at the end (Python) or check inside the loop (C++/Java).
- `ord(c) - ord('0')` is the standard trick to convert a digit char to its integer value.

Codex Solution

1. Intuition

This problem is mainly about **parsing the string from left to right** while following strict rules:

1. Ignore leading spaces.
2. Read at most one optional sign.
3. Read digits until the first non-digit.
4. Clamp the result if it goes outside the 32-bit signed integer range.

The key pattern is a **single-pass scanner**.

We do not need to examine the whole string after digits stop mattering. As soon as we hit a non-digit after processing optional whitespace/sign, parsing ends.

To handle overflow safely, we can check **before adding the next digit** whether the number would exceed the allowed range.

The valid range is:

```
[-2^31, 2^31 - 1]
= [-2147483648, 2147483647]
```

Notice that the negative side allows one more absolute value:

```
abs(-2147483648) = 2147483648
```

So the allowed magnitude depends on the sign.

2. Approach

Pseudocode

```
function myAtoi(s):
    INT_MAX = 2147483647
    INT_MIN = -2147483648

    i = 0
    n = length of s

    // Step 1: skip leading spaces
    while i < n and s[i] == ' ':
        i++

    // Step 2: read optional sign
    sign = 1
    if i < n and s[i] is '+' or '-':
        if s[i] == '-':
            sign = -1
        i++

    // Step 3: read digits
    result = 0

    if sign is positive:
        limit = INT_MAX
    else:
        limit = 2147483648

    while i < n and s[i] is digit:
        digit = integer value of s[i]

        // Check if result * 10 + digit would exceed limit
        if result > (limit - digit) / 10:
            if sign is positive:
                return INT_MAX
            else:
                return INT_MIN

        result = result * 10 + digit
```

```
i++
```

```
// Step 4: apply sign  
return sign * result
```

3. Python Solution

```
class Solution:
    def myAtoi(self, s: str) -> int:
        # 32-bit signed integer boundaries
        INT_MAX = 2**31 - 1      # 2147483647
        INT_MIN = -2**31        # -2147483648

        n = len(s)
        i = 0

        # -----
        # Step 1: Ignore leading whitespace.
        # The problem only asks us to ignore the space character ' ',
        # not all possible whitespace characters.
        # -----
        while i < n and s[i] == ' ':
            i += 1

        # -----
        # Step 2: Determine the sign.
        # Default sign is positive.
        # If we see '+' or '-', consume exactly one sign character.
        # -----
        sign = 1

        if i < n and (s[i] == '+' or s[i] == '-'):
            if s[i] == '-':
                sign = -1
            i += 1

        # -----
        # Step 3: Convert consecutive digits into an integer.
        # We store the absolute value in result.
        # -----
        result = 0

        # For positive numbers, maximum allowed magnitude is INT_MAX.
        # For negative numbers, maximum allowed magnitude is 2147483648,
        # because INT_MIN = -2147483648 is valid.
        limit = INT_MAX if sign == 1 else 2**31

        while i < n and s[i].isdigit():
            digit = ord(s[i]) - ord('0')

            # -----
            # Overflow check before multiplying by 10.
            #
            # We want to know if:
            #
            #     result * 10 + digit > limit
            #
            # Rearranged safely:
            #
            #     result > (limit - digit) // 10
            #
            # If this condition is true, adding this digit would exceed
            # the 32-bit signed integer range.
            # -----
            if result > (limit - digit) // 10:
                return INT_MAX if sign == 1 else INT_MIN

            result = result * 10 + digit
            i += 1

        # -----
```

```
# Step 4: Apply the sign and return the parsed integer.
# If no digits were read, result is still 0, so this returns 0.
# -----
return sign * result
```

4. Complexity Analysis

Let n be the length of the string.

Time Complexity

$O(n)$

We scan each relevant character at most once:

- leading spaces,
- optional sign,
- consecutive digits.

In the worst case, the whole string is processed once.

Space Complexity

$O(1)$

We only use a constant number of variables:

- index i ,
- sign,
- result,
- integer limits.

No extra data structures are used.

5. Alternative Approaches

1. Parse Then Clamp

In Python, integers are unbounded, so we could first build the full integer and clamp afterward:

```
result = min(INT_MAX, max(INT_MIN, result))
```

This is simpler, but less language-independent because in languages like C++ or Java, the integer could overflow while being built.

2. Regular Expression

We can use regex to extract the optional sign and digit prefix:

```
import re

match = re.match(r"^\s*([+-]?\d+)", s)
```

Then convert and clamp.

This is concise but usually not preferred in interviews because the problem is testing manual parsing.

3. Finite State Machine

We can model parsing with states such as:

```
START -> SIGNED -> IN_NUMBER -> END
```

This is more general and clean for complex parsers, but for this problem, a direct scanner is simpler.